Cloud Computing: Assignment Mini Project

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

1 Overview

I wanted to assign you a task on LLMs that utilises cloud computing concepts. But as you know, deploying LLMs on your low-end laptops is non-trivial. So you'll be using knowledge distillation over a larger model, the teacher model (which is still not large enough, probably 2 to 4 B parameters), into an extremely small model (the student model, let's say 70 million parameters). And then you will use this smaller model for the deployment. Yes, it will be quite a useless model, but the quality is not our concern.

Knowledge distillation (KD) is a model-training technique in which a smaller *student model* learns to reproduce the outputs or behaviour of a more capable *teacher model*. The objective is to obtain a more computationally efficient model that retains useful task-specific capabilities while reducing the resources required for inference.

So dig a little deeper on your own first and explore how KD works. How can you apply KD practically? I have put some links in the subsequent sections for kickstarting, but do not limit yourself to these only.

You'll be using AI and no one can stop you. **But do use it with your eyes wide open.**

In this project, you will apply **sequence-level knowledge distillation** to a command-parsing task. You will first run a teacher model to generate responses for a collection of natural language commands. You will then use these responses as training targets to fine-tune a smaller language model, and then deploy the resulting model as a service on a local Kubernetes-based cloud environment.

2 Learning Objectives

If you do it diligently (even after using AI), then you should be able to: 1. Explain the

motivation, basic mechanism, and limitations of knowledge distillation.

2. Prepare and process a teacher-generated dataset using reproducible data-processing work flows.

3. Fine-tune and evaluate a lightweight language model on CPU-only hardware. 4. Store

model artefacts in object storage and retrieve them during application deployment. 5.

Containerise an inference service and deploy it using Kubernetes.

6. Configure resource requests and limits, health probes, autoscaling, rolling updates, and recovery procedures.

7. Apply subnet allocation and reason about stateless network access-control rules.
8. Automate software validation and model evaluation using a CI/CD (Continuous Integration and Continuous Delivery) workflow.
9. Evaluate a deployed service using latency, throughput, error rate, resource consumption, and cost estimates.

3 Some Preliminary Resources

Go through these before implementation for the background. As I mentioned earlier, do not limit yourself to these resources alone.

1. **Introduction to knowledge distillation.** PyTorch, *Knowledge Distillation Tutorial*. This tutorial introduces the teacher–student framework and demonstrates how a student can learn from a teacher's output distributions. Its main example uses classification, so it provides conceptual background rather than the exact training procedure required in this project.
   https://docs.pytorch.org/tutorials/beginner/knowledge_distillation_tutorial. html
2. **The method you will actually use.** Kim and Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Read Sections 1 to 3. The student is trained directly on the teacher's generated outputs, which is exactly what you will do.
   https://aclanthology.org/D16-1139/
3. **Distillation for language generation.** Chen et al., *Distilling Knowledge Learned in BERT for Text Generation*, ACL 2020. You may focus on the motivation, the teacher–student setup, and the experimental design rather than the mathematical details.
   https://aclanthology.org/2020.acl-main.705/
4. **A practical example of language-model distillation.** Hugging Face TRL, *Distillation Trainer*. This documentation describes a more advanced method that trains a student to match a teacher's token-level output distribution. It is recommended for comparison; you are not required to implement this method.
   https://huggingface.co/docs/trl/distillation_trainer
5. **Fine-tuning a small causal language model.** Hugging Face Transformers, *Causal language modeling* task guide. This is the closest practical guide to Task 2.
   https://huggingface.co/docs/transformers/tasks/language_modeling
6. **Student model documentation.** EleutherAI, *Pythia-70M Model Card*. This resource describes the model, intended research use, limitations, and licence.
   https://huggingface.co/EleutherAI/pythia-70m
7. **Local Kubernetes environment.** The k3d documentation explains how to run a lightweight Kubernetes environment locally using containers.
   https://k3d.io/stable/

8. **Object storage.** SeaweedFS provides an open-source storage system with an

S3-compatible interface.
https://github.com/seaweedfs/seaweedfs

9. **Kubernetes resource monitoring.** The Kubernetes resource metrics documentation explains the metrics infrastructure used by CPU-based autoscaling.
https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipel ine/

NOTE: You are not expected to reproduce the research papers or implement a general-purpose distillation algorithm from scratch. See how it is done practically and use it for your task.

4 Application Description

You are building a small command parser, the kind of thing that sits inside a voice assistant. It takes one short English command and returns a JSON object that says what the user wants (the **intent**) and the key details (the **slots**). For example:

Input

```
set an alarm for seven thirty am tomorrow
```

Output

```
{"intent": "alarm_set", "date": "tomorrow", "time": "seven thirty am", "place": null}
```

Every output is one JSON object with exactly four keys: intent, date, time, and place. Nothing more, nothing less.

The intent is one of these eight:

Intent Meaning and an example command

alarm_set Create an alarm: *set my alarm for six am tomorrow* alarm_query Ask about existing alarms: *what alarms do i have set for today* calendar_set Add an event or reminder: *add running to my calendar for thursday* calendar_query Ask about scheduled events: *what are my plans for the month of may* calendar_remove Delete an event: *delete my appointment tomorrow* weather_query Ask about the weather: *what is the weather in london* datetime_query Ask the date or the time: *what time is it in tokyo* recommendation_events Ask for events to attend: *what events are happening in los angeles*

The three slots follow three simple rules:

1. Copy the value exactly as it appears in the command. Do not convert it. *seven am* stays *seven am*; it does not become *07:00*.
2. Copy the shortest phrase that carries the meaning, without words like *at*, *on*, *in*, or *for*.
3. If the command does not mention a slot, it is null. Never invent a value.

One more example: *what is the weather in london* gives {"intent": "weather_query", "date": null, "time": null, "place": "london"}.

 This is a controlled demonstration, not a

general-purpose chatbot.

5 Provided Materials

The student_kit folder contains the materials listed below. The same kit is provided to all students.

```
student_kit/
  data/
    commands_pool.jsonl 3,000 commands (text only)
    validation.jsonl 100 labelled commands, for your own evaluation schema.json the
    four keys, the eight intents, the slot rules rate_card.json hypothetical prices for the
    cost exercise
  tools/
    score.py scores model outputs against labelled data locustfile.py Locust
    load test for your service
    nacl_sim.py checks NACL rules against test packets
  network/
    packets_sample.jsonl six test packets, for checking your NACL file
```

5.1 The dataset

The commands are drawn from the SLURP dataset (Bastianelli et al., *SLURP: A Spoken Language Understanding Resource Package*, EMNLP 2020, https://aclanthology.org/2020.emnlp-main.588/), released under the CC BY 4.0 licence. Only the eight intents listed above are included.

- **commands_pool.jsonl**: one command per line, {"id": "C0001", "text": "..."}. The file contains 3,000 lines. Some commands appear under different IDs by design.
- **The teacher replies**: not in the kit. You generate them yourself in Task 1.
- **validation.jsonl**: 100 commands with their correct labels, {"id": "V001", "text": "...", "intent": "...", "date": ..., "time": ..., "place": ...}. Use it to evaluate your models. **Never train on it.**

A **hidden test set** contains 300 additional commands. It is not provided to students and will be used to evaluate the submitted model.

5.2 The scripts

Run score.py or nacl_sim.py with -h to view the available options. Both require only Python 3. The load test uses Locust (https://locust.io), an open-source load-testing tool.

score.py scores model outputs.

```
python tools/score.py predictions.jsonl data/validation.jsonl [--out report.json] [-- min-validity 95]
```

Your predictions file has one line per command, {"id": "V001", "prediction": "<the raw text your

model produced>"}. The score.py reports six numbers, all in percent:

- json_validity: the output is valid JSON with exactly the four keys; •

intent_accuracy: the intent is correct;

- date_accuracy, time_accuracy, place_accuracy: that slot is correct (letter case and extra spaces are ignored; null must match null);
- full_frame: the intent *and* all three slots are correct together. This is the strictest number.

Example:

Gold Data:

{"id": "V001", "text": "remove the upcoming calendar event", "intent": "calendar_remove", "date": null, "time": null, "place": null}
{"id": "V002", "text": "need an alarm set for sunday at five in the morning", "intent": "alarm_set", "date": "sunday", "time": "five", "place": null} {"id": "V003", "text": "give me the time at washington right now", "intent": "datetime_query", "date": null, "time": null, "place": "washington"} {"id": "V004", "text": "what time does mondays meeting start", "intent": "calendar_query", "date": "mondays", "time": null, "place": null}

Predictions:

{"id": "V001", "prediction": "{\"intent\": \"calendar_remove\", \"date\": null, \"time\": null, \"place\": null}"}
{"id": "V002", "prediction": "{\"intent\": \"alarm_set\", \"date\": \"friday\", \"time\": \"five\", \"place\": null}"}
{"id": "V003", "prediction": "{\"intent\": \"alarm_set\", \"date\": \"tomor"} {"id": "V004", "prediction": "{\"intent\": \"calendar_set\", \"date\": \"mondays\", \"time\": null, \"place\": null}"}

In this example:

1. V001: fully correct.
2. V002: intent correct, but the date is wrong (“friday” instead of “sunday”). 3. V003: broken JSON (the output was cut off), so it counts as invalid and wrong on everything. 4. V004: valid JSON with the right slots, but the wrong intent.

score.py therefore returns:

```
json_validity 75.0 (3 of 4 parse correctly)
intent_accuracy 50.0 (V001, V002)
date_accuracy 50.0 (V001, V004)
time_accuracy 75.0
place_accuracy 75.0
full_frame 25.0 (only V001 is fully correct)
```

With --min-validity 95, the script exits with an error if JSON validity is below 95%. Use this

option in the CI pipeline.

The load test uses Locust with the file tools/locustfile.py. Install Locust with pip install locust, then run, from the kit folder:

```
locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m --host
    http://localhost:8080 --csv results/loadtest
```


This keeps 4 users sending POST /parse requests for 2 minutes, using the commands in validation.jsonl. A request is a failure if it does not return HTTP 200 within 30 seconds. You get:

- Locust's own summary, and results/loadtest_stats.csv, with the number of requests and failures, the median (50%) and 95% latency, and requests per second;
- results/loadtest_headers.json, with how many successful requests each replica served (from the X-Served-By header) and each model version served (from X-Model-Version). Locust cannot count headers by itself; locustfile.py adds this.

Locust exits with status 1 if any request failed. That is normal; read the numbers. For the tests in Task 4 that need load while you change something, use a longer --run-time. **The same command will be used during assessment.**

nacl_sim.py checks NACL rules.

```
python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check
```

It reads your rules, decides *allow* or *deny* for each packet the way a real NACL does (lowest rule number first, first match wins, deny if nothing matches), and with --check compares the result with the expected answer. The help text (-h) describes the rule file format. Assessment will use a hidden set of 20 packets.

6 Project Requirements

- This is an individual project. All students must complete the same tasks using the provided kit.
- Use open-source software and free tools only. No paid cloud, no paid inference API.
- Use the fixed teacher model phi3.5 (Phi-3.5-mini-instruct, 3.8 billion parameters), served with Ollama (https://ollama.com/library/phi3.5). Set temperature to 0 and seed to 42. All students must use these settings. You must write the prompt.
- Store the teacher's replies exactly as generated. Do not edit them manually.
- Train only on the teacher-generated replies. Do not substitute manually created labels, outputs from another model, or data from the validation set.
- Use EleutherAI/pythia-70m, a 70-million-parameter English model released under the Apache 2.0 licence. Assess its suitability for this task.

• Use no more than 1,500 training examples: the first 1,500 records in the cleaned dataset. •

Training and deployment may be performed separately to reduce memory use on your laptop.

• Store model weights in object storage. Do not include them in the service container image or commit them to Git.

• Do not store credentials, keys, or passwords in the repository.

7 Tasks and Assessment

The project consists of seven tasks worth 100 marks in total. Each task includes a marking table that defines the assessment criteria. Assessment will be based on the commit tagged final. Review the criteria before starting.

7.1 Task 1: Object Storage and Parallel Teacher Labelling (12 marks)

1. Create a local Kubernetes cluster with k3d, from a configuration file you keep at cluster/k3d.yaml.
2. Deploy SeaweedFS in a namespace called data. Create two buckets, datasets and models, and upload commands_pool.jsonl to datasets.
3. Run the teacher with Ollama, either on your laptop or inside the cluster; that choice is yours. Write your teacher prompt in prompts/teacher_prompt.txt. The prompt must ask for the four-key JSON described in Section 4. Labelling 3,000 commands on a CPU may take several hours. Begin this task early.
4. Write a worker, in worker/, that reads two environment variables, SHARD_INDEX (starting from 0) and SHARD_COUNT. It takes the commands whose line number (counting from 0) leaves remainder SHARD_INDEX when divided by SHARD_COUNT. For each of those, it sends the command to the teacher with your prompt and writes one line, {"id": "...", "text":
"...", "teacher_raw": "<the teacher's exact reply>"}, to its own object, for ex ample datasets/teacher/shard-0.jsonl.
5. Run at least two workers at the same time, in a namespace called pipeline, each with a different SHARD_INDEX.
6. Write worker/clean.py. It reads all the shard outputs, removes the bad records listed below, and writes the rest to datasets/v1/train.jsonl, together with its SHA-256 checksum in datasets/v1/train.sha256. Keep the records in id order. Remove:

   (a) replies that are not valid JSON;

   (b) replies whose keys are not exactly intent, date, time, and place;

   (c) replies whose intent is not one of the eight;

(d) replies with a slot value that is neither null nor text that appears in the command; (e) duplicate commands (keep the one with the smallest id).

7. In your report, give the counts: input records, rejected for each of reasons (a) to (d), duplicates, and records kept. Explain your sharding in half a page or less.

How it is marked

What I check Marks Cluster created from cluster/k3d.yaml, with SeaweedFS running in data	2
Both buckets exist; datasets holds commands_pool.jsonl, and prompts/teacher_prompt.txt is in the repository	2
At least two workers ran in pipeline with different SHARD_INDEX values, each writing its own object of raw teacher replies	2
Every command id appears in exactly one shard output: no gaps, no repeats	2
Running clean.py on the shard outputs must produce the checksum recorded in train.sha256	2
Counts table complete, and input = (a) + (b) + (c) + (d) + duplicates + kept	2

7.2 Task 2: Training the Student (Sequence-Level Distillation) (20 marks)

Fine-tune EleutherAI/pythia-70m on the first 1,500 records of your train.jsonl, using Py Torch and Hugging Face Transformers on the CPU. Build each training example like this:

```
Command: <the command>
JSON: <the teacher's JSON reply>
```

The model learns to produce the teacher's reply, given the command. That is sequence-level distillation: the teacher's output is your target. You do not need the teacher's logits, and you do not need the teacher again after Task 1, except for step 6 below.

1. Put the random seed, learning rate, batch size, number of epochs, maximum sequence length, and library versions in training/config.yaml.
2. Make training run with one command, make train, reading only that file. Also support make train SUBSET=50, which trains on the first 50 records only (This option is used to verify that training runs). Save the training loss for every epoch in training/loss.csv.

3. Save the trained model and tokenizer to the bucket models under models/v1/. Publish the same files as a GitHub Release asset.
4. Compute a SHA-256 checksum of the model files and save it as models/v1/model.sha256.
5. Train a second version (for example, one more epoch or another seed) and store it under models/v2/. This version is required for Task 4.
6. Run score.py on validation.jsonl for three models: the teacher (same prompt and settings as in Task 1), the original untrained Pythia-70M, and your trained student. Keep all three raw outputs and score reports in results/, and put the numbers side by side in your report. Save the teacher's score report as results/teacher_validation.json. Discuss what changed.
7. In one page or less, explain the difference between sequence-level and logit-based distillation, why logit-based distillation is not possible in this project, and one limitation of what you did.

Report all results accurately. The student model is expected to make errors; do not fabricate results.

How it is marked

What I check	Marks
Reproducible training (6)	
config.yaml records all six items	2
make train SUBSET=50 runs to the end on my machine	2
loss.csv has one value per epoch, and the seed is fixed in the code	2
Model artefacts (4)	
models/v1/ and models/v2/ each hold a model and tokenizer in object storage	2
The checksum of your GitHub Release asset matches models/v1/model.sha256	2
Evaluation (6)	
Raw outputs and score reports for all three models are in results/, and your report table matches them	2
On my hidden test set, JSON validity of your service is at least 95%	2
On my hidden test set, your full-frame accuracy is at least half of your teacher's full-frame accuracy in results/teacher_validation.json (2), or above 0% (1)	2
Explanation (4): 1 mark for each point stated correctly	
(a) what sequence-level distillation trains on; (b) what logit-based distillation trains on; (c) why logit-based is not possible here; (d) one real limitation of your approach	4

7.3 Task 3: The Inference Service in a Container (14 marks) Write a web service, in service/, that listens on port 8000 and has three endpoints:

- POST /parse, with body {"text": "..."}. On success, it returns HTTP 200 and the four-key JSON object, and nothing else in the body. It also sets two response headers:

X-Model-Version (your MODEL_VERSION) and X-Served-By (the Pod name).

- GET /healthz returns HTTP 200 as long as the process is alive.
- GET /readyz returns HTTP 503 until the model is loaded, and HTTP 200 after that.

Additional requirements:

- All configuration comes from four environment variables: MODEL_URI, MODEL_VERSION, S3_ENDPOINT, and MAX_NEW_TOKENS.
- At start-up, the service downloads the model and tokenizer from object storage. Credentials come from a Kubernetes Secret.
- The service checks every model output against schema.json. If the output is not valid, it returns HTTP 500 with the body {"error": "invalid_model_output"}. It never returns broken JSON with HTTP 200.
- Add one more environment variable, FAULT_INJECT. When it is 1, the service replaces the model's output with the text not json before checking it. This lets me test your validation.
- Build the image with pinned versions for every dependency, and without the model weights. Keep the image at 2 GB or less. (Hint: the default PyTorch wheels include GPU libraries you do not need.)

Then send 50 requests, one after another, to the container running with a CPU limit of 1 core and a memory limit of 1 GiB. Report the peak memory you saw. If your laptop cannot complete this test, provide supporting evidence and document the changes made.

How it is marked

What I check	Marks
API and validation (4)	
The three endpoints and both headers behave exactly as described	2
With FAULT_INJECT=1, /parse returns HTTP 500 with the exact error body	2
Model loading and configuration (4)	
The model is downloaded from object storage at start-up, and the image contains no weights	2
All four variables are used, and credentials come only from a Secret	2
Container (3)	
The image builds from a fresh clone with one command	1
Every dependency version is pinned	1
The image is 2 GB or less	1
Running under limits (3)	
50 requests complete at 1 CPU and 1 GiB with no out-of-memory kill	2
Peak memory reported, with the command output as evidence	1

7.4 Task 4: Kubernetes Deployment, Scaling, and Reliability (22 marks)

Deploy the service to your cluster:

1. In a namespace called app, create a Deployment, a Service, and an Ingress that sends /parse to the Service. (k3s already comes with the Traefik Ingress controller.)
2. Start with two replicas. Give the container CPU and memory requests and limits.
3. Add a liveness probe on /healthz and a readiness probe on /readyz.
4. Add a HorizontalPodAutoscaler (HPA): minimum 1 replica, maximum 2, target CPU utilisation 70%. (k3s already includes metrics-server.) Write down any limitation of your local cluster.
5. **Rolling update:** while the load test is running, change MODEL_VERSION and MODEL_URI from v1 to v2. Then roll back with kubectl rollout undo.
6. **Recovery:** while the load test is running, delete one Pod, and show that Kubernetes brings the replica count back.
7. **Cold start:** delete all the Pods, show from the logs that the new Pods download the model from object storage, and report the median time from Pod creation to Ready over three runs.
8. **Load balancing:** with exactly two Ready replicas, run the load test with the standard command through the Ingress, and report how many requests each replica served.

Your cluster runs on one laptop, probably as a single node. This setup does not demonstrate fault tolerance across multiple machines. State this limitation clearly.

How it is marked

What I check	Marks
Namespace, Deployment, Service, and Ingress present, and /parse answers through the Ingress	4
Requests and limits set (2); the Service has no ready endpoints while the model is still loading (2)	4
HPA as specified (2); evidence of scaling from 1 to 2 replicas under load and back down to 1 (2)	4
Rolling update with at most 0.5% failed requests (2); after rollback, X-Model-Version shows v1 again (2)	4
Replica count restored (1); at most 2% failed requests during the recovery test (1)	2
Three cold-start times and their median, with log evidence of the model download	2
In the load-balancing run, each replica served at least 30% of the successful requests	2

7.5 Task 5: Subnetting and Network Access Control (10 marks) You have the

address block 10.20.0.0/16. Divide it into four subnets with these requirements:

Subnet	Hosts needed
app	1,000
public	250
data	100
management	20

1. Allocate from the largest requirement to the smallest. Start at 10.20.0.0, place each subnet right after the previous one, and use the smallest prefix that fits each requirement.
2. For each subnet, give the network address, prefix length, usable host range, broadcast address, and number of usable hosts. Use the usual rule: usable hosts = $2^{32-\text{prefix}} - 2$. Put the table and your working in network/subnets.md.
3. Write network/nacl.json, the NACL for the public subnet. The load balancer lives there. Your NACL must do exactly this:
   - **Inbound:** block *everything* from 198.51.100.0/24 (a known abusive range), even on web ports; allow TCP 80 and TCP 443 from anywhere; allow TCP 22 (SSH) only from the management subnet; allow the replies coming back from the app subnet. Nothing else.
   - **Outbound:** allow TCP 8000 to the app subnet; allow the replies going back to clients anywhere. Nothing else.
   - Assume that clients use ephemeral ports 1024 to 65535 for their side of a

connection. 4. Check your file with nacl_sim.py and packets_sample.jsonl.

5. In half a page or less, explain why a stateless NACL needs separate rules for reply traffic, and why the order of your rules matters for the 198.51.100.0/24 block.

How it is marked

What I check	Marks
Subnet table: 1 mark for each subnet with all five values correct	4
NACL: 4× (fraction of my 20 hidden packets decided correctly), rounded to the nearest 0.5	4
Explanation: a NACL does not remember connections, so replies need their own rule (1); a deny rule only works if its number is lower than the allow rule it must override (1)	2

7.6 Task 6: Reproducibility, Testing, and CI/CD (10 marks)

Keep everything (code, tests, training and evaluation scripts, configuration, Kubernetes manifests) in your GitHub repository. Then:

1. make up builds and deploys the whole system from a fresh clone, in at most three commands. make down deletes the cluster.

2. Write a GitHub Actions workflow with these stages, in this order: linting; unit tests (at least three); a model evaluation that downloads your model from the GitHub Release and runs score.py on validation.jsonl; and a quality gate, score.py --min-validity 95.

3. Show the gate doing its job: run the workflow once with a deliberately bad model (the untrained Pythia-70M is a good choice) and put the link to the failed run in your report.

4. Add a secret-scanning step (for example, gitleaks). It must report nothing.

5. Write AI_USE.md: which AI tools you used, for what, and how you checked what they gave you.

How it is marked

What I check	Marks
Linting stage (1); unit-test stage with at least three tests (1); evaluation stage using the Release asset (2)	4
make up works from a fresh clone in at most three commands (2); make down deletes the cluster (1)	3
Link to a run that fails at the quality gate	2
AI_USE.md present, and the secret scan reports nothing	1

7.7 Task 7: Performance, Cost, and Review (12 marks)

Run the load test with the standard command against your deployment and report:

- p50 and p95 latency;
- throughput, in requests per second;
- error rate;
- CPU and memory of each Pod during the test (from kubectl top pods); •

cold-start time (from Task 4).

Cost. Using rate_card.json, estimate the cost of 1,000 requests, showing every step:

$$\text{cost per 1,000 requests} = \frac{\text{hourly cost of all running replicas}}{\text{requests served per hour}} \times 1000$$

Each replica is billed for its CPU limit and memory limit for every hour it runs. This is an estimated cost; no actual cloud charge is incurred on your laptop.

SLO. Define one Service Level Objective, for example “99% of requests succeed”. State the target and the window (one load-test run). Work out the error budget in number of requests, and how much of it you used.

Review. Fill in this table. Each “Evidence” cell must include a measured value.

Component or concern	and re covery Security and cost **How it is**	**marked**
Data and storage Model quality Latency and throughput Resource utilisation Reliability		**Evidence (a measured number)** **Limitation you found** **Improvement you propose**

What I check	Marks
All five measurements reported, with the Locust output files attached (2); on my machine, error rate at most 1% and p95 at most 5,000 ms (2)	4
Cost correct for the given rates (3); right method but an arithmetic slip (1); otherwise (0)	3
SLO with a target and a window (1); error budget worked out correctly in requests (1)	2
Review table: 0.5 marks for each row with all three cells filled and a measured number as evidence	3

8 Submission Requirements

1. **Hand Written Reflection:** Upload the scanned copy of your reflections. Reflect on what you have learnt, what challenges you faced, how you overcame them, what could have been done better, observations, findings, etc., in a 3-page document. It should have your GitHub link for the assignment, your name and roll number. The filename should be <roll_number>.pdf. Follow this naming convention strictly. Files that do not follow this naming convention, including those with extra spaces or additional text, will receive zero marks.
2. **GitHub repository:** code, tests, training and evaluation scripts, Dockerfile, Kubernetes manifests, network files, and CI workflow.
3. **Model artefacts:** the trained model, tokenizer, version information, and checksums, as a GitHub Release asset. Do not commit model files to Git.

4. **Evidence:** logs, score.py outputs, Locust outputs, and command outputs, in results/. 5.

README.md: the commands to build, deploy, test, and tear down your project.

6. **Tag:** tag your final commit as final. Only that commit will be assessed. Commits made after the deadline will not be considered.

7. **Report:** report/report.pdf, at most eight pages (references and appendices excluded), with one section per task. Cover your architecture, decisions, setup, results, limitations, and improvements. Keep measured results and assumptions clearly separate.

9 Additional References

Official documentation you will probably need along the way:

- Hugging Face Transformers: https://huggingface.co/docs/transformers/index, and *Text generation*: https://huggingface.co/docs/transformers/llm_tutorial
- k3d, *Using Config Files*: https://k3d.io/stable/usage/configfile/
- K3s, *Networking Services* (Traefik and the built-in load balancer): https://docs.k3s.io/networking/networking-services
- Boto3, *Amazon S3 examples* (set endpoint_url to your SeaweedFS address): https://boto3.amazonaws.com/v1/documentation/api/latest/guide/s3-examples.html
- Kubernetes, *Deployments*: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes, *Service*: https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes, *Liveness, Readiness, and Startup Probes*: https://kubernetes.io/docs/concepts/workloads/pods/probes/
- Kubernetes, *Secrets*: https://kubernetes.io/docs/concepts/configuration/secret/
- Kubernetes, *Horizontal Pod Autoscaling*: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
- Kubernetes, *Performing a Rolling Update*: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/
- Docker, *Building best practices*: https://docs.docker.com/build/building/best-practices/
- Docker, *Resource constraints*: https://docs.docker.com/engine/containers/resource_constraints/
- FastAPI, *FastAPI in Containers*: https://fastapi.tiangolo.com/deployment/docker/
- GitHub Actions, *Quickstart*: https://docs.github.com/en/actions/get-started/quickstart
- Gitleaks: https://github.com/gitleaks/gitleaks
- AWS VPC User Guide, *Network ACLs*: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

- Google SRE Book, *Service Level Objectives*: https://sre.google/sre-book/service-level-objectives/